



Tecnológico de Monterrey

**Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Puebla**

Materia: Implementación de robótica inteligente

Clave: TE3002B.501

Grupo: 501

Profesor:

Rigoberto Cerino Jiménez
Juan Manuel Ahuactzin Larios
Alfredo García Suárez
Cesar Torres Huitzil

Alumnos

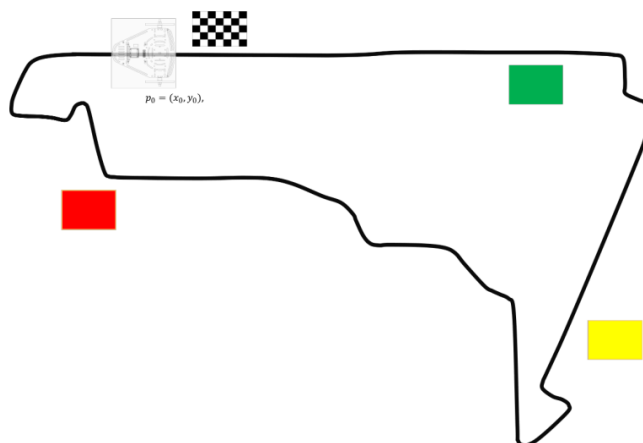
Omar Perez Dominguez | A01738306
Ezequiel Luna Trejo | A01738153
Antonio Méndez Rodríguez | A01738269

16 de Mayo De 2026

Resumen

El presente proyecto consistió en el diseño, desarrollo e implementación de un sistema de navegación autónoma y visión para la plataforma robótica diferencial Puzzlebot, en el ecosistema de Manchester Robotics. La arquitectura de software se estructuró de manera modular bajo el framework ROS 2 Humble, distribuyendo las tareas en nodos especializados de procesamiento embebidos en una computadora NVIDIA Jetson Nano.

El sistema implementa un algoritmo robusto de seguimiento de línea basado en visión computacional mediante el uso de un pipeline de GStreamer, umbralización adaptativa y una técnica de ventanas deslizantes (sliding windows) por momentos para calcular el error lateral y aproximar el radio de curvatura en tiempo real. En paralelo, se integró una capa de toma de decisiones para interpretar la señalización de un semáforo dinámico mediante segmentación en el espacio de color HSV. Esta última fue calibrada para mitigar los efectos de sobreexposición lumínica. El control del vehículo se rige por un lazo cerrado proporcional-integral (PI) con técnicas de anti-windup y saturación para regular los comandos cinemáticos. El robot es capaz de modular su velocidad lineal de forma autónoma: reduce el avance preventivamente ante curvas cerradas o una luz amarilla, realiza un paro total y obligatorio ante una luz roja manteniendo el estado de detención incluso tras la pérdida del estímulo visual y reanuda la marcha al confirmar la luz verde. Ante perturbaciones severas que provoquen la pérdida total de la línea, el controlador activa una rutina de rescate basada en la memoria del último error conocido y en estimaciones de pose por odometría para reincorporar al robot de manera segura al circuito.



Objetivos

Objetivo general

Desarrollar e implementar un sistema de navegación autónoma y percepción visual para la plataforma robótica móvil diferencial Puzzlebot bajo el framework ROS 2 Humble, integrando algoritmos de procesamiento de imágenes y un lazo de control cerrado para realizar el seguimiento continuo de una línea guía y la toma de decisiones dinámicas basada en la interpretación de un semáforo.

Objetivos Específicos

- Procesamiento robusto de línea: Diseñar e implementar un algoritmo de visión computacional basado en umbralización adaptativa y ventanas deslizantes (sliding windows) para extraer el error lateral y aproximar la curvatura de la pista de forma inmune a los cambios de iluminación del entorno.
- Segmentación: Calibrar los rangos de segmentación en el espacio de color HSV para la detección precisa de las luces roja, amarilla y verde de un semáforo, acoplando un filtro de persistencia temporal por conteo de fotogramas para mitigar falsos positivos y destellos por sobreexposición lumínica.
- Control cinemático de lazo cerrado: Desarrollar un controlador Proporcional-Integral (PI) que module las velocidades lineal y angular del chasis en función del error lateral detectado, incorporando funciones de saturación y técnicas de anti-windup para garantizar un movimiento suave y proteger los componentes electrónicos del chasis.
- Rutina automática de rescate: Programar una estrategia de seguridad basada en estimaciones de pose por odometría y la memoria del último error conocido, permitiendo al robot ejecutar una búsqueda angular controlada para reincorporarse autónomamente al circuito en caso de pérdida de línea.
- Validación experimental en pista: Validar el desempeño del sistema mediante pruebas físicas en escenarios dinámicos, asegurando un error de posicionamiento aceptable, paros totales obligatorios ante la luz roja y reducciones preventivas de velocidad ante curvas cerradas y luz amarilla.

Introducción

1. Procesamiento de imágenes en tarjetas embebidas

El procesamiento de imágenes en tiempo real demanda una alta capacidad de cómputo y una gestión eficiente de la memoria, restricciones que se vuelven críticas al trabajar en plataformas móviles. Las tarjetas embebidas modernas, como la NVIDIA Jetson Nano, resuelven este desafío integrando arquitecturas que combinan una CPU multinúcleo con una GPU dedicada.

A diferencia de los procesadores convencionales, estos sistemas permiten la ejecución en paralelo de operaciones matriciales densas, características de la visión computacional. En este proyecto, la Jetson Nano actúa como el cerebro central del Puzzlebot, permitiendo ejecutar simultáneamente la adquisición de video, el filtrado espacial, la segmentación en espacios de color y el cálculo de la odometría, manteniendo un consumo energético y térmico adecuado para un robot de dimensiones reducidas. [1]

2. Interconexión entre Jetson Nano y la cámara mediante GStreamer

Para mitigar la latencia en la captura de frames y maximizar la tasa de refresco (FPS), la interconexión entre el sensor óptico de la cámara y la Jetson Nano se realiza mediante un pipeline de GStreamer optimizado para hardware de NVIDIA.

A través del elemento `nvarguscamerasrc`, el sistema accede directamente al backend de la cámara utilizando la memoria compartida del sistema (`memory:NVMM`). Esto permite realizar la conversión de formatos de color (de la codificación nativa del sensor NV12 a formato espacial BGR apto para OpenCV) mediante el módulo de hardware `nvvidconv`. Al delegar esta transformación al hardware dedicado, se libera a la CPU de la Jetson Nano de tareas repetitivas de copia de memoria, garantizando un flujo constante de 20 FPS hacia el bucle principal de ejecución de ROS 2. [2]

3. Algoritmos de percepción para el seguimiento de líneas y trayectorias

El núcleo perceptivo del robot se divide en dos regiones de interés (ROI) dentro de la imagen capturada para procesar de manera independiente la señalización vial y la trayectoria:

- Detección de semáforos: Mediante la segmentación en el espacio de color HSV (Hue, Saturation, Value), el sistema es inmune a las variaciones de brillo absoluto, aislando las componentes cromáticas de las luces roja, amarilla y verde. Para filtrar el ruido y los falsos positivos por sobreexposición lumínica, se implementan operaciones morfológicas de apertura y un filtro de persistencia temporal basado en el conteo secuencial de fotogramas.
- Seguimiento de línea (Ventanas deslizantes): En la región inferior de la imagen se aplica un umbralizado adaptativo gaussiano (`cv2.adaptiveThreshold`) que extrae los bordes de la línea guía de forma dinámica frente a reflejos o sombras en el suelo. Posteriormente, la trayectoria se discretiza horizontalmente mediante la técnica de ventanas deslizantes (*Sliding Windows*), donde se calculan los momentos espaciales de primer orden (centroides $M10/M00$) de cada franja. Estos puntos permiten calcular analíticamente el error lateral del robot respecto al eje de la pista y estimar el radio de curvatura analizando la tendencia geométrica de los centroides más lejanos. [3]

4. Robustez en sistemas de procesamiento de imágenes

La robustez en visión computacional se define como la capacidad del algoritmo para mantener un desempeño aceptable ante perturbaciones severas del entorno. En pistas reales de exteriores o interiores, factores como baches, cambios abruptos de iluminación, texturas ruidosas en el suelo o la pérdida temporal de la línea representan desafíos críticos.

Este proyecto implementa la robustez de manera multinivel:

1. A nivel de píxel: Mediante desenfoques gaussianos adaptativos para suavizar texturas indeseadas en el suelo.
2. A nivel algorítmico: Mediante el almacenamiento del último error conocido de la línea.
3. A nivel de sistema: Si el procesamiento visual experimenta un fallo total por la ausencia de la línea guía, el control del robot conmuta automáticamente a una rutina de rescate. Esta rutina rompe temporalmente la dependencia visual y recurre a un estimador probabilístico basado en odometría cinemática diferencial, utilizando las velocidades de los encoders de las ruedas (WR , WL) calculadas en tiempo real para estimar la orientación (θ) a través de transformaciones angulares cuaterniónicas.

Con este estado síncrono, el Puzzlebot ejecuta un giro angular controlado para reincorporarse de forma segura al circuito. [4]

5. Lazo de control cinemático cerrado (PI)

Finalmente, para traducir las métricas de visión en movimientos físicos, la arquitectura cierra el lazo mediante un Controlador Proporcional-Integral (PI) de velocidad angular. La acción proporcional (K_p) corrige de forma inmediata las desviaciones del error lateral detectado por las ventanas deslizantes, mientras que la acción integral (K_i) mitiga el error en estado estacionario producido por desalineaciones físicas en el chasis diferencial.

Para evitar la inestabilidad del robot ante correcciones acumuladas bruscas, el lazo incorpora límites estrictos de saturación de velocidad angular y técnicas de anti-windup, acotando la acumulación de la integral cuando el actuador alcanza sus límites físicos. La velocidad lineal se modula de manera proactiva: se reduce ante la proximidad de curvas cerradas (radios de curvatura pequeños) o advertencias de semáforo amarillo, y efectúa un paro total e irreversible ante una luz roja, garantizando una navegación suave, segura y precisa. [5]

Solución del problema

En esta sección se detalla el diseño, la estructura algorítmica y la implementación de los nodos modulares desarrollados bajo el framework ROS 2 Humble. Esta arquitectura permite la separación de tareas, optimizando el rendimiento computacional de la plataforma NVIDIA Jetson Nano y facilitando la depuración del sistema en tiempo real.

Nodo de odometría

El nodo de odometría es el componente encargado de realizar la estimación del estado y la pose del Puzzlebot en el plano cartesiano (x, y, θ) a una frecuencia de operación fija de 100 Hz (período de 0.01 s). Esta información es fundamental para que el controlador maestro conozca la ubicación relativa del robot respecto a su punto de inicio y pueda ejecutar la rutina de recuperación cuando no se detecte la línea guía.

Configuración y suscripción de sensores

Para garantizar una lectura de datos de alta fidelidad y libre de delays introducidos por las colas de mensajes en la red, el nodo se suscribe a los tópicos de los encoders de los motores derecho (VelocityEncR) e izquierdo (VelocityEncL) utilizando un perfil de calidad de servicio especializado para hardware denominado qos_profile_sensor_data. Este perfil prioriza la entrega inmediata de las velocidades angulares del motor descartando paquetes antiguos.

```
20     self.create_subscription(  
21         Float32, 'VelocityEncR', self.encR_callback, qos_profile_sensor_data  
22     )  
23     self.create_subscription(  
24         Float32, 'VelocityEncL', self.encL_callback, qos_profile_sensor_data  
25     )  
26     self.odom_pub = self.create_publisher(Odometry, 'odom', 10)  
27     self.create_timer(0.01, self.run)  
28  
29     def encR_callback(self, msg): self.wr = msg.data  
30     def encL_callback(self, msg): self.wl = msg.data
```

Inicialización de constantes físicas

El nodo define formalmente las dimensiones geométricas del chasis diferencial, las cuales fueron calibradas experimentalmente para mitigar la deriva o el drift acumulativo propio de la integración numérica:

- Radio de las ruedas (r): 0.05 m.
- Distancia entre llantas (l): 0.1615 m.

```
10 def __init__(self):
11     super().__init__('odom')
12     self.x, self.y, self.th = 0.0, 0.0, 0.0
13     self.r, self.l = 0.05, 0.1615
14     self.wr, self.wl = 0.0, 0.0
15     self.V, self.Omega = 0.0, 0.0
16     self.last_time = self.get_clock().now()
17     self.first = True
18
```

Lazo de integración y cálculo dinámico de dt

A diferencia de los esquemas tradicionales con diferenciales de tiempo fijos, este nodo calcula el tiempo transcurrido exacto (dt) entre cada iteración del temporizador utilizando el reloj interno del sistema en nanosegundos:

$$dt = (tiempo_{actual} - tiempo_{anterior}) * 10^{-9}$$

Esta estrategia dota al sistema de robustez algorítmica, ya que si la CPU de la Jetson Nano experimenta un pico de procesamiento debido a la capa de visión artificial y retrasa sutilmente la ejecución del lazo de odometría, el cálculo del dt absorbe esta variación real, evitando desfases en la integración de la pose.

```
32 def run(self):
33     current_time = self.get_clock().now()
34     dt = (current_time - self.last_time).nanoseconds * 1e-9
35     if self.first:
36         self.first, self.last_time = False, current_time
37     return
```


Tras obtener el diferencial temporal, se ejecuta la conversión cinemática y el método de Euler para actualizar el estado del robot. Asimismo, se aplica la normalización angular trigonométrica para asegurar que el ángulo de orientación permanezca acotado de manera segura:

```
38
39     vr, vl = self.r * self.wr, self.r * self.wl
40     self.V = 0.5 * (vr + vl)
41     self.Omega = (vr - vl) / self.l
42     self.x += self.V * np.cos(self.th) * dt
43     self.y += self.V * np.sin(self.th) * dt
44     self.th += self.Omega * dt
45     self.th = np.arctan2(np.sin(self.th), np.cos(self.th))
46     self.last_time = current_time
47
```

2. Capa de percepción visual: Nodo de visión

Este nodo es el componente encargado de transformar el flujo de video crudo en variables geométricas e indicadores de estado legibles para el sistema de decisión. Se ejecuta de a una frecuencia de 20 FPS mediante un temporizador interno, lo que garantiza un procesamiento síncrono, predecible y de baja latencia en el entorno del PuzzleBot.

2.1. Adquisición de video mediante pipeline GStreamer

Para mitigar la alta carga computacional que sufren las tarjetas embebidas al procesar video en tiempo real, se configuró un pipeline nativo de GStreamer en la arquitectura de la Nvidia Jetson Nano a través de la API de OpenCV.

- Mecanismo de operación: En lugar de capturar video por los canales estándar de la CPU, el comando “nvarguscamerasrc” extrae los fotogramas directamente de la memoria de video de la GPU.
- Justificación: La imagen se reduce a una resolución de 640*480 píxeles y se convierte al formato BGR de forma directa en el hardware de la tarjeta. Esto elimina las transferencias lentas de memoria entre la CPU y la GPU (cuellos de botella), asegurando que el nodo procese frames en tiempo real sin retrasos acumulativos.

2.2. Segmentación por regiones de interés (ROI)

Dentro del bucle principal (process_loop), cada frame capturado se divide simétricamente a la mitad horizontal mediante operaciones de indexación de matrices:

- **roi_top** (Mitad superior): Enfocada al horizonte visual, espacio donde se ubican físicamente los semáforos instalados en la pista.
- **roi_bot** (Mitad inferior): Enfocada estrictamente a la superficie del suelo por donde transita la plataforma.

Justificación: Al recortar la imagen en dos zonas de interés específicas, se descartan elementos irrelevantes del entorno (como techos, paredes o luces del techo). Esto reduce a la mitad el número de píxeles que deben analizar los algoritmos subsecuentes, disminuyendo drásticamente el costo computacional global del nodo.

2.3. Subsistema de clasificación de semáforos (procesar_semaforo)

El objetivo de este bloque es aislar e identificar de manera limpia y estable los tres estados posibles del semáforo (Rojo, Amarillo, Verde). Para lograrlo, la región **roi_top** se somete al siguiente pipeline de procesamiento:

- **Conversión al espacio de color HSV:** La imagen original en BGR se transforma al espacio HSV (Hue, Saturation, Value). A diferencia del modelo BGR (donde cualquier cambio de luz altera los tres canales simultáneamente), el espacio HSV separa la información del color puro (Tono/Hue) de la intensidad de la iluminación (Brillo/Value). Esto dota al sistema de inmunidad frente a variaciones de luz ambiental, sombras o destellos en exteriores.
- **Filtrado morfológico de apertura:** Con el fin de limpiar el ruido de alta frecuencia en las máscaras binarias, se aplica una operación de apertura morfológica utilizando un kernel estructurante de 5*5. Esta técnica ejecuta primero una erosión (eliminando puntos blancos pequeños y aislados provocados por ruido en la cámara) seguida de una dilatación (restaurando el tamaño del objeto real detectado).
- **Criterio de área máxima e inmunidad a la distancia:** El algoritmo localiza los contornos y selecciona únicamente el área del contorno más grande. Para evitar que el robot reaccione ante objetos irrelevantes o ruidos lejanos del mismo color en el fondo, cada estado tiene asignado un umbral mínimo de píxeles en un diccionario de configuración. Si el contorno de color detectado no supera dicho umbral, el estímulo se descarta inmediatamente.
- **Filtro de persistencia temporal:** En ambientes de prueba reales, un reflejo momentáneo puede hacer que la cámara interprete un color erróneo durante un único

frame. Para evitar que el PuzzleBot efectúe paradas bruscas o cambios de velocidad por falsos positivos, se implementó una lógica de consistencia temporal basada en contadores. Un cambio en el estado del semáforo solo se valida y publica en el tópico `/traffic_state` si el color candidato se detecta de manera idéntica durante 5 frames consecutivos, actuando como un filtro pasa-bajas digital para la toma de decisiones visuales.

2.4. Subsistema de guiado y extracción de la línea (`procesar_linea`)

Este bloque se encarga de extraer la geometría de la pista a partir de la `roi_bot` para transformarla en los vectores de error cinemático que requiere el controlador del robot.

- **Atenuación de ruido por desenfoque gaussiano:** La sub-imagen del suelo se convierte a escala de grises y se somete a un filtro de desenfoque Gaussiano empleando un kernel de 9×9 . Esta operación actúa eliminando los detalles diminutos y las imperfecciones de alta frecuencia del pavimento (como la textura rugosa del piso, grietas o líneas de unión entre lozas), evitando que estas irregularidades sean confundidas con la línea de guía.
- **Umbralizado adaptativo gaussiano local:** En pistas de prueba reales, el uso de un umbral estático de binarización suele fallar cuando el robot pasa de una zona iluminada a una zona con sombras. Para solucionar esto, se implementó un umbralizado adaptativo inverso con una ventana de vecindario amplia de 115×115 píxeles. Este algoritmo calcula un umbral dinámico individual para cada píxel basándose en la media ponderada gaussiana de su entorno inmediato menos una constante de ajuste.
- **Técnica de múltiples centroides por ventanas deslizantes:** Para interpretar la trayectoria de la línea en el espacio (tanto en rectas como en curvas), la imagen binaria se segmenta verticalmente en 6 franjas horizontales equidistantes, las cuales se analizan secuencialmente desde la parte inferior de la pantalla (lo más cercano al parachoques del robot) hacia la parte superior (el horizonte visual):
 1. Para cada franja horizontal se calculan sus momentos espaciales estadísticos.
 2. Se evalúa el momento de orden cero (M_{00}), el cual representa el área de masa blanca dentro de esa franja. Si el valor supera los 300 píxeles, la ventana se valida como parte de la línea real y se descarta que sea ruido disperso.

3. Se calcula la coordenada horizontal del centroide (C_x) dividiendo el momento de primer orden entre el momento de masa (M_{10}/M_{00}). Este método permite seguir la trayectoria con alta precisión, incluso si la línea física se encuentra parcialmente desgastada o fragmentada en la pista.
- Cálculo de variables de control exclusivas: A partir de la distribución espacial de los centroides obtenidos, el nodo genera la telemetría que consume la capa de control:
Error lateral: Se calcula midiendo la distancia ortogonal entre el centro geométrico de la pantalla y el centroide de la primera ventana (la base). Esta métrica describe con exactitud qué tan desfasado se encuentra el eje del PuzzleBot respecto al centro de la pista en el instante actual.

Pseudo-radio de curvatura: Se evalúa midiendo la diferencia horizontal absoluta entre el centroide de la ventana base y el centroide de la ventana más lejana en el horizonte. Cuando el robot se desplaza en una línea recta, ambos centroides se alinean verticalmente en el mismo eje; al aproximarse una curva, el centroide lejano se desplaza hacia un extremo con mayor velocidad. El inverso de esta diferencia matemática genera un indicador predictivo que le permite a la capa de control anticipar la severidad de una curva antes de que el cuerpo físico del robot ingrese a ella.

Finalmente, estas tres variables críticas (`error_lateral`, `curvatura`, `linea_encontrada`) se empaquetan en un mensaje estructurado de tipo `Float32MultiArray` y se publican sincronamente en el tópico `/line_features`.

3. Capa de decisión y control: Nodo de control

Este nodo actúa como el cerebro del robot. Se ejecuta a una frecuencia alta de 50 Hz y coordina las acciones de control físico basadas en la combinación de datos visuales y odométricos.

- Suscripción multitópico: Monitorea de forma síncrona los estados de `/traffic_state`, `/line_features` y `/odom` para actualizar constantemente sus variables internas.
- Función `control_loop()` (Máquina de estados de navegación):

Caso 1: Estado de detención por semáforo

Si semaforo_estado == 0 (Luz Roja), las consignas cinemáticas se imponen inmediatamente en cero (cmd.linear.x = 0.0, cmd.angular.z = 0.0) y se vacía el acumulador integral del control para prevenir sobreimpulsos cuando se reanuda el movimiento. El robot prioriza la seguridad ante cualquier estímulo de la pista.

Caso 2: Navegación guiada por lazo cerrado PI

Si la línea guía es visible (linea_encontrada == 1.0), el puzzlebor se deja controlar por la ley de control Proporcional-Integral:

1. Acción integral con anti-windup: Acumula el error lateral multiplicándolo por el paso de tiempo:

$$error_integral = error_integral + error_lateral * dt$$

Para evitar la saturación destructiva del término integral (efecto windup) ante errores sostenidos, el valor es acotado entre [-200.0, 200.0]

2. Ecuación del controlador angular: Calcula la velocidad angular necesaria para corregir el rumbo:

$$u(t) = Kp * error_lateral + Ki * error_integral$$

Donde las ganancias calibradas son $Kp = 0.004$ y $Ki = 0.0005$.

3. Saturación dinámica de actuadores: Para salvaguardar la integridad de la electrónica y evitar pérdidas de tracción bruscas, se imponen limitadores: la velocidad angular se acota a 0.8 rad/s y la velocidad lineal máxima a 0.15 m/s.
4. Modulación de velocidad lineal: La velocidad base se adapta según el semáforo (Verde = 0.15 m/s, Amarillo = 0.08 m/s). Adicionalmente, si el radio_curvatura cae por debajo de un umbral crítico de 400, se aplica un factor de atenuación proporcional a la severidad de la curva:

$$factor = \max(0.4, \frac{Radio\ de\ curvatura}{400})$$

$$velocidad\ lineal = velocidad\ base * factor$$

Esto reduce de forma preventiva la velocidad lineal del vehículo a un mínimo seguro del 40% en curvas cerradas para evitar derrapes.

Caso 3: Maniobra autónoma de rescate por odometría

Si el nodo de visión reporta la pérdida de la línea guía (`linea_encontrada == 0.0`), el robot transiciona de inmediato al estado "RECUPERANDO".

1. Fijación de la pose de pérdida: Al ingresar a la rutina por primera vez, el robot memoriza su orientación angular actual (`yaw_perdida = yaw_actual`) provista por la odometría y resetea los términos integrales de control.
2. Búsqueda angular controlada: Utilizando la función `wrap_angle()`, el nodo calcula el desplazamiento angular absoluto respecto al punto donde falló la visión:

$$\Delta \theta = |\text{wrap_angle}(\text{yaw actual} - \text{yaw perdida})|$$

3. Estrategia de giro basada en memoria: Mientras `Delta_theta` sea menor al límite de seguridad establecido de (`math.radians(60)`), el robot mantiene una velocidad lineal nula y aplica una rotación suave y constante de 0.3 rad/s. El sentido del giro está determinado por el signo del `ultimo_error_conocido`, forzando al robot a buscar la línea en la dirección lógica donde fue vista por última vez.
4. Parada de emergencia absoluta: Si el barrido angular supera los 60° sin que las ventanas deslizantes vuelvan a detectar la línea, el sistema de rescate aborta de manera segura, deteniendo por completo los motores para evitar que la plataforma colisione o se desvíe indefinidamente de la zona de pruebas.

Resultados

En esta sección se presentan las evidencias experimentales que validan la integración del sistema de navegación con la capa de toma de decisiones basada en visión computacional. Los resultados obtenidos demuestran la capacidad del robot para completar trayectorias mientras responde de manera autónoma a señales de tráfico dinámicas presentes en el entorno.



Prueba unitaria de detección de color en el semáforo

Antes de habilitar el lazo de control de movimiento del chasis, se ejecutó una prueba unitaria estática con el propósito de validar el comportamiento de la capa de percepción visual en el nodo SuperVisionNode. Esta fase experimental permitió comprobar que la calibración de los rangos de matiz, saturación y valor (HSV) fuera capaz de clasificar correctamente los estados del semáforo ante una exposición lumínica directa del semáforo.

Durante esta prueba estática se identificó un fenómeno físico relevante: la intensidad de la luz verde inunda el fondo de la escena y genera un gradiente de contraste falso en el suelo. Este reflejo es detectado por el umbral adaptativo en la región inferior, lo que provoca que las ventanas deslizantes calculen centroides con un desplazamiento artificial en forma de zig-zag, arrojando un error lateral estacionario.

📺 Deteccion de semaforo.mp4

Prueba integrada de seguimiento de carril y respuesta a semáforo

Se realizó una prueba de validación del sistema completo en un escenario dinámico continuo. El objetivo fue evaluar la capacidad del nodo controlador para ejecutar simultáneamente el seguimiento del carril guía negro mediante ventanas deslizantes y reaccionar de forma síncrona a las señales lumínicas del semáforo procesadas por la cámara en la mitad superior de la imagen.

📺 Seguimiento de carril y semaforo.mp4

En la mitad inferior de la imagen, el sistema ejecuta el algoritmo de umbralización adaptativa. Se observa que los centroides calculados por momentos (puntos verdes) y el vector de trayectoria (líneas rojas) registran una desviación angular en forma de zig-zag hacia la izquierda del carril real. Este comportamiento constituye un hallazgo técnico crítico: la alta luminosidad del semáforo se refleja sobre la superficie pulida del suelo, creando un falso gradiente de contraste que el umbral adaptativo procesa como parte de la línea negra. A pesar de esta oscilación transitoria en las ventanas superiores, la base del algoritmo mantiene el anclaje sobre el carril real, permitiendo un error lateral controlado

📺 Seguimiento de carril y semaforo otra vista.mp4

Pruebas de recorrido completo en la pista

En esta fase experimental se ejecutaron los recorridos integrales sobre la pista física para evaluar el desempeño combinado y la sincronización en tiempo real de los nodos de odometría, visión computacional y control. Los videos analizados demuestran la capacidad de la plataforma Puzzlebot para resolver la pista, responde de forma autónoma a las perturbaciones del entorno y a la señalización dinámica del semáforo.

Prueba de la pista 1.mp4

Se presenta la segunda prueba del recorrido completo en la pista física. Durante esta fase experimental, el desempeño del algoritmo de seguimiento de carril resultó altamente satisfactorio y preciso; sin embargo, se constató la necesidad de prestar especial atención a las variaciones de la iluminación ambiental. A pesar de que las pruebas se ejecutaron en un espacio interior cerrado y con luz de un foco encendido, la infiltración de luz solar directa proveniente del exterior introdujo ruido óptico transitorio, provocando pequeñas fallas intermitentes que el sistema computacional del chasis logró compensar y reajustar de forma autónoma. Adicionalmente, el intenso brillo emitido por el semáforo LED se reflejó sobre la superficie del suelo, generando perturbación visual. Pese a la presencia simultánea de estas interferencias lumínicas del entorno real, la robustez del lazo cerrado de control y los filtros de percepción permitieron al Puzzlebot estabilizar su marcha y completar con éxito la totalidad del circuito diseñado.

Prueba de la pista 2.mp4

Conclusiones

A partir de la ejecución de las pruebas experimentales y el análisis de los datos recolectados, se concluye que los objetivos planteados fueron alcanzados satisfactoriamente mediante la implementación de una arquitectura modular en ROS 2 Humble. El cumplimiento exitoso de las metas se fundamenta en la integración efectiva de los componentes especializados que se desarrollaron:

- **Navegación autónoma y control:** Se logró un seguimiento preciso del carril guía gracias a la acción conjunta del controlador proporcional-integral (PI) y el estimador de odometría, manteniendo al chasis alineado con la pista y permitiendo desaceleraciones dinámicas óptimas ante la presencia de curvas cerradas. El error de posición se mantuvo dentro de los parámetros de tolerancia física definidos.
- **Visión computacional y toma de decisiones:** El algoritmo de percepción visual demostró una alta robustez para discriminar las señales de tráfico en entornos dinámicos. Esto se atribuye al rediseño estratégico de los límites en el espacio de color HSV y, fundamentalmente, al filtro de persistencia temporal de fotogramas, el cual neutralizó los cambios de estado erráticos provocados por el ruido lumínico.
- **Seguridad vial:** El sistema respetó de forma estricta la regla de detenerse ante una luz roja y permanecer inmóvil hasta detectar explícitamente la luz verde.

Limitaciones e imprevistos identificados

- **Efectos de la fricción estática:** Las no linealidades inherentes a los motores del Puzzlebot y la fricción del suelo requirieron una compensación manual mediante el ajuste de un umbral mínimo de velocidad angular para evitar el estancamiento del chasis ante errores pequeños.
- **Ruido lumínico ambiental:** Aunque las pruebas se realizaron en un espacio cerrado con iluminación artificial constante, la infiltración de luz solar externa y el reflejo del LED del semáforo sobre el suelo brillante generaron un gradiente de contraste falso. Esta perturbación distorsionó temporalmente la posición de los centroides calculados por las ventanas deslizantes superiores, provocando ligeras oscilaciones en forma de zig-zag que, si bien fueron toleradas por el control, añadieron esfuerzo a los actuadores.

Propuestas de mejora y soluciones futuras

- Restricción dinámica de la región de búsqueda (ROI): Modificar el nodo de visión para que la posición de búsqueda de cada ventana deslizante dependa del centroide calculado por la ventana inmediatamente inferior. Esta restricción espacial evitará que el algoritmo salte hacia reflejos o tarjetas electrónicas externas a la pista.
- Optimización geométrica de la señal visual: Robustecer la detección del semáforo implementando algoritmos de búsqueda basados en la geometría circular de las luces, tales como la Transformada de Hough para círculos. Esta técnica independizará la toma de decisiones de la simple segmentación por color, reduciendo drásticamente los falsos positivos en entornos con fondos complejos o sobreexpuestos.

Referencia:

- [1] NVIDIA Corporation. (2020). *NVIDIA Jetson Nano Developer Kit User Guide*. <https://developer.nvidia.com/embedded/jetson-nano>
- [2] GStreamer Open Source Project. (2024). *GStreamer Multiplatform Multimedia Framework Documentation*. <https://gstreamer.freedesktop.org/documentation/>
- [3] Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson. <https://books.google.com/books?id=P99QDwAAQBAJ>
- [4] Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2011). *Introduction to Autonomous Mobile Robots* (2nd ed.). MIT Press. <https://books.google.com/books?id=atw6CwAAQBAJ>
- [5] Åström, K. J., & Hägglund, T. (2006). *Advanced PID Control*. International Society of Automation. https://books.google.com/books?id=48Y_AQAAIAAJ
- [6] Manchester Robotics Ltd. (2026). TE3002B: Intelligent Robotics Implementation [Repositorio de GitHub]. Recuperado de https://github.com/ManchesterRoboticsLtd/TE3002B_Intelligent_Robotics_Implementation_2026
- [7] Computer Vision- Dr Cesar Torres Huitzil [GoogleColab.Google.com](https://colab.research.google.com/).
https://colab.research.google.com/drive/1e5Z82Gca0yfptq3Y2j8RO6e0oZsUmt_A#scrollTo=j9DZrxWkFgxV